

OSGi Working Group
OSGi Specification repo for
`org.osgi.service.tracker` Re-
lease 1.0.3-SNAPSHOT

September 2026



Copyright © 2000, 2026 Eclipse Foundation

LICENSE

Eclipse Foundation Specification License – v1.0

By using and/or copying this document, or the Eclipse Foundation document from which this statement is linked, you (the licensee) agree that you have read, understood, and will comply with the following terms and conditions:

Permission to copy, and distribute the contents of this document, or the Eclipse Foundation document from which this statement is linked, in any medium for any purpose and without fee or royalty is hereby granted, provided that you include the following on ALL copies of the document, or portions thereof, that you use:

- link or URL to the original Eclipse Foundation document.
- All existing copyright notices, or if one does not exist, a notice (hypertext is preferred, but a textual representation is permitted) of the form: "Copyright © [date-of-document] Eclipse Foundation, Inc. <<url to this license>>".

Inclusion of the full text of this NOTICE must be provided. We request that authorship attribution be provided in any software, documents, or other items or products that you create pursuant to the implementation of the contents of this document, or any portion thereof.

No right to create modifications or derivatives of Eclipse Foundation documents is granted pursuant to this license, except anyone may prepare and distribute derivative works and portions of this document in software that implements the specification, in supporting materials accompanying such software, and in documentation of such software, PROVIDED that all such works include the notice below. HOWEVER, the publication of derivative works of this document for use as a technical specification is expressly prohibited.

The notice is:

"Copyright © [date-of-document] Eclipse Foundation. This software or document includes material copied from or derived from [title and URI of the Eclipse Foundation specification document]."

Disclaimers

THIS DOCUMENT IS PROVIDED "AS IS," AND THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION MAKE NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NON-INFRINGEMENT, OR TITLE; THAT THE CONTENTS OF THE DOCUMENT ARE SUITABLE FOR ANY PURPOSE; NOR THAT THE IMPLEMENTATION OF SUCH CONTENTS WILL NOT INFRINGE ANY THIRD PARTY PATENTS, COPYRIGHTS, TRADEMARKS OR OTHER RIGHTS.

THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION WILL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, SPECIAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF ANY USE OF THE DOCUMENT OR THE PERFORMANCE OR IMPLEMENTATION OF THE CONTENTS THEREOF.

The name and trademarks of the copyright holders or the Eclipse Foundation may NOT be used in advertising or publicity pertaining to this document or its contents without specific, written prior permission. Title to copyright in this document will at all times remain with copyright holders.

Preface

Implementation Requirements

An implementation of a Specification: (i) must fully implement the Specification including all its required interfaces and functionality; (ii) must not modify, subset, superset or otherwise extend the OSGi Name Space, or include any public or protected packages, classes, Java interfaces, fields or methods within the OSGi Name Space other than those required and authorized by the Specification. An implementation that does not satisfy limitations (i) (ii) is not considered an implementation of the Specification and must not be described as an implementation of the Specification. "OSGi Name Space" shall mean the public class or interface declarations whose names begin with "org.osgi" or any recognized successors or replacements thereof. An implementation of a Specification must not claim to be a compatible implementation of the Specification unless it passes the Technology Compatibility Kit ("TCK") for the Specification.

Feedback

This specification can be downloaded from the OSGi Documentation web site:

<https://docs.osgi.org/specification/>

Comments about this specification can be raised at:

<https://github.com/osgi/osgi/issues>

Table of Contents

1	Introduction	5
1.1	Reader Level	5
1.2	Version Information	5
1.3	References	5
131	TRo69 Connector Service Specification	7
131.1	Introduction	7
131.2	TR-069 Protocol Primer	8
131.3	TR069 Connector	13
131.4	RPCs	23
131.5	Error and Fault Codes	26
131.6	Managing the RMT	27
131.7	Native TR-069 Object Models	28
131.8	org.osgi.service.tr069todmt	28
131.9	References	36

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

1 Introduction

This document contains a specification developed by the OSGi working group.

1.1 Reader Level

This specification is written for the following audiences:

- Application developers
- Framework and system service developers (system developers)
- Architects

This specification assumes that the reader has at least one year of practical experience in writing Java programs. Experience with embedded systems and server-environments is a plus. Application developers must be aware that the OSGi environment is significantly more dynamic than traditional desktop or server environments.

System developers require a very deep understanding of Java. At least three years of Java coding experience in a system environment is recommended. A Framework implementation will use areas of Java that are not normally encountered in traditional applications. Detailed understanding is required of class loaders, garbage collection, Java 2 security, and Java native library loading.

Architects should focus on the introduction of each subject. This introduction contains a general overview of the subject, the requirements that influenced its design, and a short description of its operation as well as the entities that are used. The introductory sections require knowledge of Java concepts like classes and interfaces, but should not require coding experience.

Most of these specifications are equally applicable to application developers and system developers.

1.2 Version Information

Each OSGi specification has an independent version, and packages defined by those specifications have their own versions as well. Specification and package versions follow semantic versioning rules.

1.3 References

- [1] *OSGi Specifications*
<https://docs.osgi.org/specification/>

DRAFT

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

131 TRo69 Connector Service Specification

Version 1.0

131.1 Introduction

This chapter provides a specification for the TR069 Connector, an assistant to a Protocol Adapter based on [1] *TR-069 Amendment 3*. A TR069 Connector provides a mapping of TR-069 concepts to/from the DMTP. It mainly handles the low level details of Object/Parameter Name to Dmt Admin URI mapping, and vice versa. TR-069 Protocol Adapter developers can use this service to simplify the use of the Dmt Admin service. The TR069 Connector service is based on the definition of a Protocol Mapping in DMTP. It is assumed that the reader understands TR-069 and has a basic understanding of the Dmt Admin service.

The examples in this specification are not from a Broadband Forum Technical Report and are purely fictional.

131.1.1 Essentials

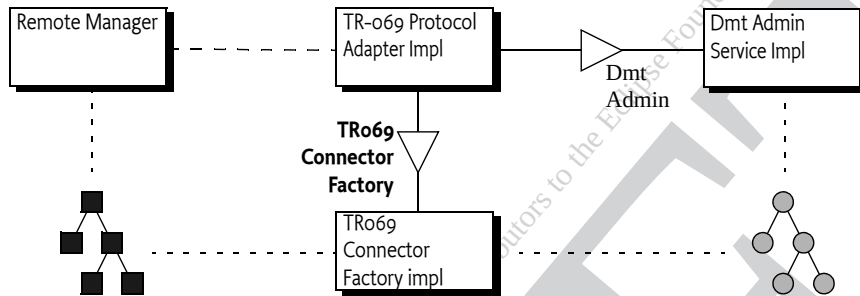
- *Connector* - Provide a TR-069 view on top of the Dmt Admin service.
- *Simplify* - Simplify the handling of data models implemented through the DMTP through the TR-069 protocol.
- *Browse* - Implement the constructs for MAP and LIST handling.
- *Native* - Provide a mechanism for Data Plugins to convey conversion information to the Protocol Adapter so that native TR-069 object models can be implemented as Data Plugins.

131.1.2 Entities

- [TRo69ConnectorFactory](#) - Provides a way to create a [TRo69Connector](#) that is bound to an active Dmt Session.
- [TRo69Connector](#) - Created by [TRo69ConnectorFactory](#) on a Dmt Session; provides methods that help in using the TR-069 namespace and RPCs on a Dmt Admin DMTP.
- [ParameterValue](#) - The value of a parameter, maps to the TR-069 ParameterValueStruct.
- [ParameterInfo](#) - Information about the parameter, maps to the TR-069 ParameterInfoStruct.
- DMTP - The Device Management Tree as available through the Dmt Admin service.

Figure 131.1

TR-069 Entities



131.1.3

Synopsis

A TR-069 Protocol Adapter first creates a Dmt Session on the node in the DMT that maps to an object model that should be visible to the TR-069 Management Server. A Protocol Adapter can choose to map a whole sub-tree or it can create a virtual object model based on different nodes, this depends on the implementation of the Protocol Adapter.

When a TR-069 RPC arrives, the Protocol Adapter must parse the SOAP message and analyze the request. In general, an RPC can request the update or retrieval of multiple values. The Protocol Adapter must decompose these separate requests into single requests and execute them as a single unit. If the request is a retrieval or update of a data model maintained in the Dmt Admin service then the Protocol Adapter can use a TR069 Connector to simplify implementing this request. The TR069 Connector Factory service can be used to create an instance of a TR069 Connector that is based on a specific Dmt Session.

The TR069 Connector maps the Object or Parameter Name to a URI and perform the requested operation on the corresponding node. The name-to-URI conversion supports the LIST and MAP concepts as defined in ???.

The TR069 Connector handles conversion from the Dmt Admin data types to the TR-069 data types. There is a default mapping for the standard Dmt Admin formats including the comma separated list supported by TR-069. However, Data Plugins that implement TR-069 aware object models can instruct the TR069 Connector by providing specific MIME types on the Meta Node.

Objects can be added and deleted but are, in general, not added immediately. These objects are lazily created when they are accessed. The reason is that TR-069 does not support the concept of a session with atomic semantics, a fact leveraged by certain object models in the DMT. Therefore, adding an object will assign a instance id to an object but the creation of the object is delayed until the object is used.

After all the requests in an RPC are properly handled the TR069 Connector must be closed, the Dmt Session must be closed separately.

Errors are reported to the caller as they happen, if a Dmt Admin service error is fatal then the Dmt Session will be closed and it will be necessary to create a new TR069 Connector.

131.2

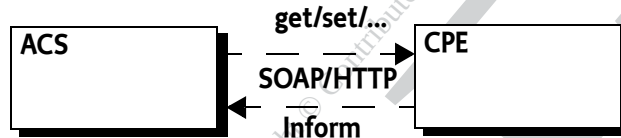
TR-069 Protocol Primer

The [6] *Broadband Forum* is an organization for broadband wire-line solutions. They develop multi-service broadband packet networking specifications addressing interoperability, architecture, and management. Their specifications enable home, business and converged broadband services, encompassing customer, access and backbone networks. One of the specifications of the Broadband Forum is the *Technical Report No 69*, also called TR-069, a specification of a management model.

131.2.1 Architecture

[1] *TR-069 Amendment 3* is a technical report (Broadband Forum's specification model) that specifies a management protocol based on [4] *SOAP 1.1* over HTTP. The TR-069 technical report defines a number of mandatory Remote Procedure Calls (RPCs) that allow a management system, the Auto-Configuration Server (ACS), to discover the capabilities of the Consumer Premises Equipment (CPE) and do basic management. This model is depicted in Figure 131.2.

Figure 131.2 TR-069 Reference Architecture



In TR-069, the CPE is always initiating the conversation with the ACS though the ACS can request a session.

Inside the CPE there is a Protocol Adapter that implements the TR-069 RPCs. These RPCs read and modify the objects models present in the CPE. There is usually a mechanism that allows the different modules in the CPE to contribute a management object to the Protocol Adapter so that the Protocol Adapter does not require knowledge about highly specialized domains.

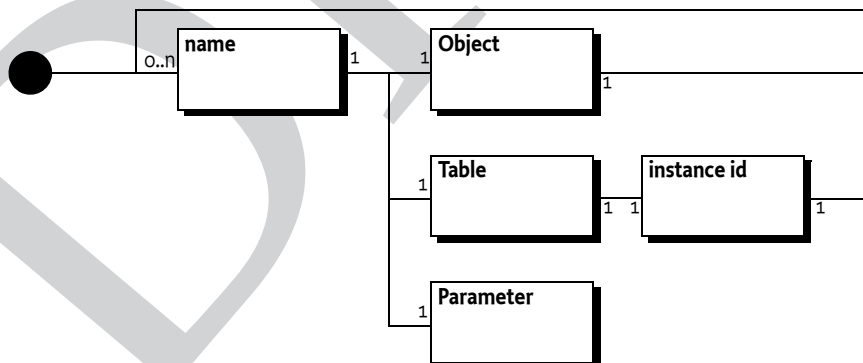
[2] *TR-106 Amendment 3* specifies object model guidelines to be followed by all TR-069-enabled devices as well as a formal model to document these object models.

131.2.2 Object Model

The object model of TR-069 consists of *objects* that contain *parameters* as well as *tables* that contain objects. TR-106 says:

- *Object* - A named collection of parameters and/or other objects.
- *Parameter* - A name-value pair.
- *Table* - An enumeration of objects identified by an instance id.

Figure 131.3 Type Model TR-069



Objects can also occur in tables, in that case the object name is suffixed with an *instance id*. An object that has no instance id is a singleton, with an instance id they are referred to as *tables*. In the Broadband Forum technical reports tables end in the special suffix {i}, the instance id.

This provides the following structural definition for this specification:

```
named-value ::= NAME ( object | table | parameter )
```

```

object      ::= named-value +
table       ::= ( instance object ) *
parameter   ::=
instance    ::= INTEGER > 0

```

TR-069 talks about partial paths and parameter names. In this specification, a *name* is reserved for the short relative name used inside an object, also called the *local name*. The term *path* is reserved for the combination of object names, table names, and instance ids that are separated by a full stop ('.') and used to traverse an instance model.

```

path        ::= parameter-path | object-path | table-path
segment     ::= NAME '.' ( instance '.' ) ?
object-path ::= segment +
table-path  ::= segment * NAME '.' // expect INTEGER next
parameter-path ::= object-path NAME
instance-path ::= table-path instance '.'

```

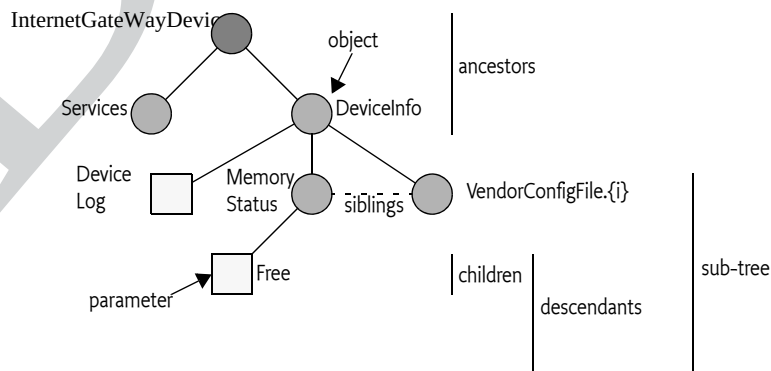
In this specification the following terms are used consistently:

- *Object* - Refers to a named type defining a certain set of parameters, objects, and tables.
- *Table* - A list of instances for a given object.
- *Instance* - An object element in a table at a certain id.
- *Instance Id* - The integer id used to identify an instance in a table.
- *Alias* - A name chosen by the ACS that uniquely identifies an instance.
- *Singleton* - An object that is not in a table.
- *Name* - The name of an object, table, or parameter refers to the local name only and not the path.
- *Segment* - A component in a path that always ends in a full stop. A segment can contain instance ids to identify an instance.
- *Path* - A string uniquely identifying a path in the tree to either a parameter, an object, or a table.
- *Object Path* - A path that uniquely identifies an instance or a singleton. An object path must always ends in a full stop. This maps to the TR-069 concept of an ObjectName.
- *Parameter Path* - The name of the parameter preceded by the owning object. A path that does not end in a full stop is always a parameter path.
- *Table Path* - An object path that lacks the last instance id. In TR-069 this is also sometimes called a partial path. The last segment is an object path that must be followed by an instance id to address an instance.
- *Instance Path* - A path to an instance in a table

This provides a hierarchy as depicted in Figure 131.4 .

Figure 131.4

TR-069 Object and Parameter naming relative to the parameter *MemoryStatus*



131.2.3 Parameter Names

The grammars for parameter names and object names are as follows:

NAME ::= (Letter | ' _ ')
(Letter | Digit | '-' | '_' | CombiningChar | Extender)*

The productions Letter, Digit, CombiningChar, and Extender are defined in [5] *Extensible Markup Language (XML) 1.0 (Second Edition)* . The name basically supports the full Unicode character set for letters and digits (including digits for other languages), including sets for languages like Hebrew and Chinese. Examples of different parameter names are:

name // simple name
Name // case sensitive
-
_ _ _ _
ångstrom
þingsten
ΨΣΩΠ

131.2.4 Parameter Type

A parameter value can have one of the data types defined in [2] *TR-106 Amendment 3* , they are summarized in Table 131.1.

Table 131.1 TR-106 Data types

TR-106 Type	Description
object	Represents a structured type
string	A Unicode string, optionally restricted in length
int	32 bit integer
long	64 bit integer
unsignedInt	32 bit unsigned integer
unsignedLong	64 bit unsigned integer
boolean	Can have values 0 or false (false) or 1 or true (true)
dateTime	TR-069 recognizes three different date times. These three cases are differentiated in the following way: • <i>Unknown time</i> - If the time is not known. • <i>Relative time</i> - Relative time is the time since boot time. • <i>Absolute time</i> - Normal date and time.
base64	An array of bytes
hexBinary	An array of bytes

SOAP messages always provide a type for the parameter value. For example:

<ParameterValueStruct>
 <name>Parameter1</name>
 <value xsi:type="long">1234</value>
</ParameterValueStruct>

The xsi prefix refers to the <https://www.w3.org/2001/XMLSchema-instance> namespace. However, this makes not all TR-106 types well defined, for example in XML Schema base64 is called base64Bi-

nary. This specification assumes that the names and definitions in Table 131.1 and provides appropriate constants for the Protocol Adapter.

Parameters can be read-only or read-write. All writable Parameters must also be readable although security can cause certain parameters to be read as an empty string, for example passwords. Parameters can reflect configuration as well as status of the device. External causes can cause parameters to change at any time. The TR-069 protocol has the facility to call an Inform RPC to provide the ACS with a notification of changed parameters.

131.2.5

Parameter Attributes

Parameter attributes provide the meta data for a parameter. In TR-069, the attributes are used to manage notifications and access control. Each parameter in TR-069 can be watched by the ACS by setting the corresponding parameter attribute to *active* or *passive notifications*. Passive notifications are passed whenever the CPE communicates with the ACS and active notifications initiate a session. Parameters that have a notification are said to be *watched*.

Access to the parameters can be managed by setting Access Control Lists via the corresponding parameter attribute.

131.2.6

Objects and Tables

TR-106 has the concept of an *object* stored in a *table* to allow multiple instances of the same type. It is part of the object definition if it is stored in a table or not. An object cannot both appear as a table instance and as a singleton.

Each instance in the table is addressed with an integer ≥ 1 . This *instance id* is not chosen by the ACS since it can be required to create a new instance due to an external event. For example the user plugging in a USB device or starting a new VOIP session. The ACS must discover these instance ids by asking the device for the instance ids in a table.

For example, the parameter path Device.LAN.DHCPOption.4.Request refers to a parameter on a DHCPOption object that has the instance id 4. Instance ids are not sequential nor predictable. It is the responsibility of the device to choose an instance id when an object is created. Instance ids are assumed to be persistent so that the ACS can cache results from a discovery process.

Newer TR-069 objects have been given an Alias parameter. This alias uniquely identifies the table instance.

TR-069 defines a convention for a parameter that contains the number of entries in a table. Any parameter name that ends with NumberOfEntries contains the number of entries in a table with the name of the prefix in the same object. For example A.B.CNumberOfEntries provides the number of entries in the table:

A. B. C.

131.2.7

RPCs

The object model implemented in a device is accessed and modified with *RPCs*. RPCs are remote procedure calls; a way to invoke a function remotely. TR-069 defines a number of mandatory RPCs and provides a mechanism to extend and discover the set of RPCs implemented by a CPE. The mandatory RPCs are listed in in the following table.

Table 131.2 TR-069 RPCs

RPC	Description
GetRPCMethods	Return a list of RPC methods
SetParameterValues	Set one or more parameter values
GetParameterValues	Get one or more parameter values

RPC	Description
GetParameterNames	Get the parameter information for a parameter, object, or table.
SetParameterAttributes	Set parameter attributes
GetParameterAttributes	Get parameter attributes
AddObject	Add a new object to a table
DeleteObject	Delete an object from a table
Download	Download software/firmware
Reboot	Reboot the device

131.2.8 Authentication

The security model of TR-069 is based around the authentication taking place during the setup of a TLS (formerly SSL) connection. This authentication is then used to manage the access control lists via the parameter attributes.

131.2.9 Sessions and Transactions

A *session* with the ACS is always initiated by the CPE. The ACS can request a session, but it is always the CPE that starts a session by opening the connection to the ACS and then sending an Inform RPC. The session ends when the connection is closed, which happens after the ACS has informed the CPE it has no more requests.

During a session, a CPE has the requirements that parameters must not change due to other sources than the session and that the parameters are consistent with the changes. However, there is no transactionality over the session, atomicity is only guaranteed for one RPC. An RPC can consist of multiple parameter modifications that should therefore be atomically applied.

131.2.10 Events and Notifications

TR-069 sessions always start with an Inform RPC from the CPE to the ACS. This RPC contains any events and notifications for parameters that were watched. Events signal crucial state changes from the CPE to the ACS. For example, if a device has rebooted it will inform the ACS. Notifications are caused by parameter changes, the Inform RPC contains a list of events and parameters with changed values.

131.2.11 Errors

Invoked RPCs can return a fault status if errors occur during the execution of the RPC. For ACS to CPE RPCs these fault codes start at 9000, for the reverse direction they start at 8000. Each RPC defines the fault codes that can occur and their semantics in that context.

131.3 TRo69 Connector

A TR-069 Protocol Adapter must be able to browse foreign Data Plugins on the device and support native TR069 objects models implemented by a Data Plugin. As Data Plugins are available through the Dmt Admin service, the Protocol Adapter must provide a bi-directional mapping between Dmt Admin nodes and TR-069 parameters, notifications, and error codes. The mapping must enable a Data Plugin to provide a native Broadband Forum object model that limits itself to the required RPCs.

131.3.1 Role

Developers implementing the TR-069 protocol are not likely to be also experts in the Dmt Admin service. This specification therefore provides a TR069 Connector Factory service that provides an object that can map from the TR-069 concepts to the Dmt Admin concepts, supporting all the constructs defined in the ???.

The TR069 Connector only specifies a number of primitive functions to manage the DMT. Parsing the SOAP messages, handling the notifications, and splitting the requests for TR069 Connector is the responsibility of the Protocol Adapter. The reason that the TR069 Connector does not work on a higher level is that a Protocol Adapter for TR-069 will likely communicate with other subsystems in the CPE than the OSGi framework alone. Though the Dmt Plugin model is an attractive approach to implement object models, there is history. Existing code will likely not be rewritten just because it can be done better as a Data Plugin.

For example, a Data Plugin could implement the `Device.DeviceInfo` object. However, this object actually resides in the DMT at a node:

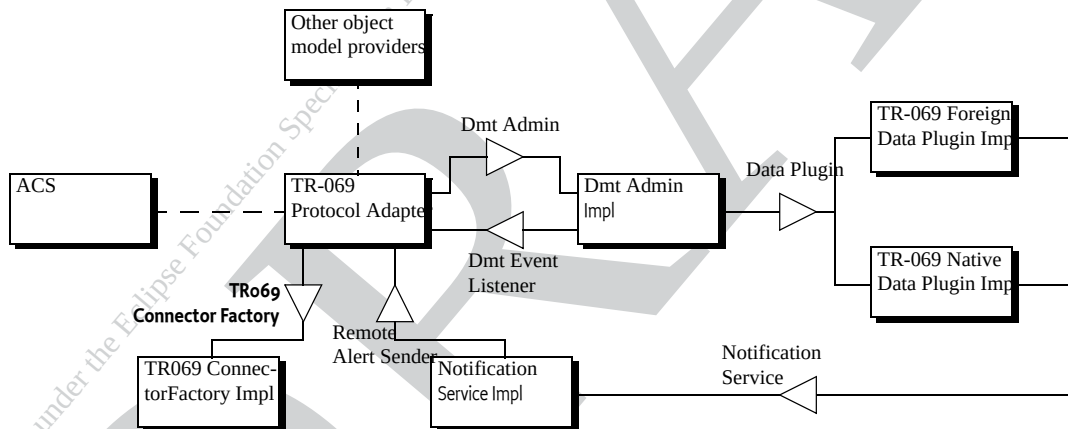
```
./TR-069/Device/DeviceInfo
```

A TR-069 Protocol Adapter will therefore be confronted with a number of data models that reside in different places. Each place provides one or more consistent data models but it is the responsibility of the TR-069 Protocol Adapter to ensure the ACS gets a consistent and standardized view of the whole. To create this consistent view it will be necessary to adapt the paths given in the RPCs. It is expected that a Protocol Adapter is required to have a certain amount of domain knowledge, for example a table, that maps TR-069 paths to their actual providers.

The basic model is depicted in Figure 131.5.

Figure 131.5

TR-069 Connector Context



The Protocol Adapter can be implemented as an OSGi Bundle or it can be implemented in native code in the device. Both architectures are viable. For certain aspects like the TR-157a3 Software Modules a certain amount of native code will be required to manage the OSGi Framework as an Execution Environment.

In an environment where the Protocol Adapter is implemented outside an OSGi Framework it will be necessary to create a link to the Dmt Admin service. This can be achieved with a proxy bundle inside the OSGi framework that dispatches any requests from the native Protocol Adapter to the functionality present in the OSGi Framework. In this specification, it is assumed that such proxies can be present. However, the examples are all assuming that the Protocol Adapter is running as a Bundle.

131.3.2

Obtaining a TR069 Connector

A TR069 Connector is associated with a Dmt Session, the [TR069ConnectorFactory](#) provides the [create\(DmtSession\)](#) method that will return a [TR069Connector](#) object. This object remains associated with the Dmt Session until the Dmt Session is closed, which can happen because of a fatal error or when the TR069 Connector Factory is unregistered or un-gotten/released. Creating a TR069 Con-

connector must not be expensive, Protocol Adapters should create and close them at will. Closing the connector must not close the corresponding Dmt Session.

The TR069 Connector must use the root of the session as its *base*. That is, their URI mapping all parameters must start from the base. For example, if the session is opened at `./TR-069` then the parameter `IGD/DeviceInfo/Manufacturer` must map to URI `./TR-069/IGD/DeviceInfo/Manufacturer`.

If a Protocol Adapter will modify the tree then it should use an atomic session for all RPCs even if the RPC indicates read-only. The reason for the atomicity is that in certain cases the lazy behavior of the TR069 Connector requires the creation of objects during a read operation. If a non-atomic session is used then the TR069 Connector must not attempt to lazily create objects and reject any `addObject(String)` and `deleteObject(String)` methods. See also *Lazy and Sessions* on page 19 .

131.3.3 Supported RPCs

The TR069 Connector supports a limited number of RPCs, and for those RPCs it only supports the singleton case. The TR069 Connector provides support for the RPCs primitives listed in the following table.

Table 131.3 Supported TR-069 RPCs

RPC	Related Method	Description
SetParameterValues	<code>setParameterValue(String,String,int)</code>	Set one or more parameter values. The connector supports setting a single value, ensuring the proper path traversal and data type conversion
GetParameterValues	<code>getParameterValue(String)</code>	Get one or more parameter values. The connector supports getting a single value, converting it to a <code>ParameterValue</code> object, which contains the value and the type.
GetParameterNames	<code>getParameterNames(String,boolean)</code>	Get the paths of objects and parameters from the subtree or children that begins at the parameter path. The TR-069 Connector supports the full traversal of the given path and the next level option.
AddObject	<code>addObject(String)</code>	Add a new object to a table. The fully supports the semantics, taking the MAP and LIST nodes into account. Node creation can be delayed until a node is really needed.
DeleteObject	<code>deleteObject(String)</code>	Delete an object from a table.

131.3.4 Name Escaping

An object or parameter path describes a traversal through a set of objects, this is almost the same model that Dmt Admin provides. The difference is that the characters allowed in a TR-069 parameter name are different from the Dmt Admin node names and that TR-069 does not support application specific parameter/object names like the Dmt Admin service does.

A path consist of a number segments, where each segment identifies a name or instance id. TR-069 names can always be mapped to Dmt Admin node names as the character set of TR-069 parameter names is restricted and falls within the character set of the Dmt Admin node names. The length of a segment could be a problem but TR-069 paths are generally limited to have a length of less than 256 bytes. This specification therefore assumes that a segment of a TR-069 path is never too long to fit in a Dmt Admin node name.

Mapping a Dmt Admin node name to a parameter name, needed for browsing, is more complicated as Dmt Admin node names allow virtually every Unicode character except the solidus (`'/' \u002F`). It is therefore necessary to escape Dmt Admin URIs into a path that is acceptable for the TR-069 protocol. It is assumed that escaping is only used in a browsing mode since native object models

will never require escaping. The TR069 Connector must return names from the `getParameterNames(String,boolean)` call that the ACS can handle, optionally show to the user, and then use to construct new paths for subsequent RPCs.

There is no obvious escape character defined in TR-069, like for example the reverse solidus (`'\u005C'`) that the Dmt Admin uses for escaping. The character for escaping is the latin small letter thorn (`'þ' \u00FE'`) because his character is highly unlikely to ever be used in a TR-069 path for a native object model, however, even if it is then it would be no problem for the escaping algorithm. The thorn is a letter, allowing it to be used as the first character in a parameter name, this allows escaping the first character.

A character in a segment that is not allowed must be escaped into the following sequence:

`þ[0-9A-Z][0-9A-Z][0-9A-Z][0-9A-Z]`

The 4 hexadecimal upper case digits form a hexadecimal number that is the Unicode for that character. Each character that does not conform to the syntax specified in *Parameter Names* on page 11 or the thorn character itself must be replaced with the escape sequence. For example, the name `3ABCþ` must be translated to:

`þ0033ABCþ00FE`

If the segment is an instance id then the segment must not be escaped. Otherwise, if the segment does not start with a Letter or underscore, then the first character must be escaped with the thorn.

Unescaping must undo the escaping. Any sequence of `þ[0-9A-Z][0-9A-Z][0-9A-Z][0-9A-Z]` must be replaced with the character with the corresponding Unicode. A thorn found without the subsequent 4 hexadecimal upper case digits must be treated as a single thorn. For readability it is best to minimize the escaping. However, any name given to the TR069 Connector that is escaped must be properly interpreted even if the unescaped string did not require escaping. For example, `þ0031þ0032þ0033` must be usable as an object instance id as the unescaped form is 123, which is a number.

A number of examples of the escaping are shown in the following table.

Table 131.4 Escaping Parameter Names

Segment	Dmt Admin Escaped	TR-069 Escaped	Notes
DeviceInfo	DeviceInfo	DeviceInfo	Most common case.
3x Hello World	3x Hello World	þ0033xþ0020Helloþ0020World	The initial digit and the spaces must be escaped in TR-069.
þorn	þorn	þornþ00FEorn	A single thorn does not require escaping as it is not followed by 4 hexadecimal digits. So both forms are valid for unescaping although escaping must deliver the þ00FE form.
application/bin	application\bin	applicationþ002Fbin	The solidus must be escaped in both.
234	234	234	A numeral does not require escaping, it is assumed to be an instance id.
234x	234x	þ003234x	A name that starts with a digit requires the first digit to be escaped.
þ00FEorn	þ00FEorn	þ00FEoþ00FEorn	It is possible to encode even already escaped names.

The TR069 Connector only accepts escaped paths and returns escaped paths. When a method returns a path it must be properly escaped and suitable as a TR-069 path.

131.3.5 Root

In general, the TR-069 Protocol Adapter is free to choose what parts of the DMT it wants to expose. A simple mapping table containing path prefixes can be used to define the handler for the given data model. However, since the intention is to allow TR-069 object models to be implemented in Dmt Admin Data Plugins there is a need to know where those plugins should reside in the DMT. This root is defined as:

. / TR-069

Any Data Plugin that wants to provide an object model in the TR-069 family of object models should provide a Data Plugin rooted at the TR-069 root. For example, a Data Plugin implementing the `InternetGatewayDevice.DeviceInfo` object should register its Data Plugin under the data Root URI `./ TR-069/ InternetGatewayDevice/DeviceInfo`

131.3.6 DMT Traversal

A path must be mapped from the TR-069 hierarchy to the Dmt Admin nodes URI. The Protocol Adapter decides the *base* in the DMT by opening the Dmt Session with a session root parameter. The TR-069 Connector must then traverse the tree from this base based on the TR-069 path. The Protocol Adapter must use the ??? for MAP and LIST nodes to traverse the DMT.

Assume that the URI of a node is requested for a given path P. The path P must be traversed from the root node. The root node can find the child, the first segment in P, and then use the same routine recursively for the remainder. This recursive routine must perform the following actions on each current node:

- If path P is empty, then this is the requested node.
- S = first segment of path P up to the first full stop.
- R = remainder of path P after the first full stop or empty if no full stop.
- If S is an alias (surrounded by '[' and ']'), replace S with the alias inside the brackets. For Dmt Admin nodes aliases are identical to normal node names.
- unescape S (replace the thorns)
- If the current node is a MAP or a LIST and S is an integer
 - Get the list L of children of the current nodes
 - If the nodes in L have an InstanceId node find the node where the InstanceId matches the segment S as integer, this becomes then the next level node N and the algorithm is repeated with path R.
 - If no next node N was found then make it the child node of the current node with the name S.
 - Repeat the algorithm with N with path R

Since each node that is traversed this way knows the node name it corresponds to it is easy to create an encoded URI for Dmt Admin.

For example, the TR-069 path:

Device.DeviceInfo.Interface.14.Connections.3.BytesSent

Assuming that Interface node is a MAP node and its children have an InstanceId node, where the WAN_1 node has an InstanceId of 14.

The Connections node is a LIST and the children have no InstanceId, therefore the name is the index. The translated URI then looks like:

Device/DeviceInfo/Interface/WAN_1/Connections/3/BytesSent

The `toURI(String,boolean)` method can take a TR-069 path and perform the substitutions. If the create parameter is true then the TR069 Connector will create *missing nodes* if possible. Missing nodes can only be created under a LIST or MAP node.

A missing node is a node that is addressed by a path but not present in the DMT. For example, the root of the session is `./TR-069` and the parameter path is `A.B.C`. If the DMT contains `./TR-069/A` but not `./TR-069/A/B` then node B is a missing node.

131.3.7

Synthetic Nodes

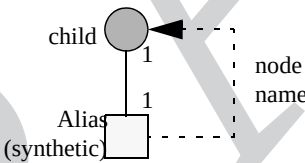
The Protocol Adapter must synthesize an Alias parameter and for any MAP or LIST node called X it must provide a sibling `XNumberOfEntries` parameter that provides the number of entries in table X.

131.3.7.1

Alias

The Alias node is a read-write parameter that must map to the actual node name of its parent. For example, `./A/B/C/Alias` must map to C. Reading it must provide the this parent's node name and writing it must rename this parent's node name. The Alias must be automatically provided on any child of a MAP node. The Alias parameter must also be returned in the result of `getParameterNames(String,boolean)` if its parent's children are included. It is not possible to convert an Alias parameter name to a URI as the Alias node is synthetic and does not exist in the DMT. The model of aliases are depicted in Figure 131.6 .

Figure 131.6 Aliases



Aliases can be used by the ACS to set the key of a MAP. For example, if a set of properties is defined as a MAP:

Name	Act	Type	Card.	S	Description
Properties [string]	Get	MAP	1	P	A Properties map
	Get Set	string	o..n	A	Key/Value
	Add Del				

An ACS can first add an object to the table. This will create an entry with a calculated instance id. However, the ACS can then rename the node with the Alias node. In pseudo code:

```
AddObject ..Properties. (returns node name = 3421)
SetParameterValue ..Properties.3421.Alias = MyKey
```

Alternatively, addressing with an alias in the parameter name would be simpler:

```
AddObject ..Properties. [MyKey]
```

131.3.7.2

Number Of Entries

TR-069 has the convention of parameters that end with `NumberOfEntries`. For example, the parameter `UserNumberOfEntries` in the object `InternetGatewayDevice` object contains the number of entries of the `InternetGatewayDevice.User` table.

The Protocol Adapter must synthesize these NumberOfEntries parameters for each MAP or LIST node. The NumberOfEntries parameter must be a sibling of the MAP or LIST node. Any such parameter must also be returned in the result of the `getParameterNames(String,boolean)` method.

131.3.8 Lazy and Sessions

In the Dmt Admin service the session plays an important role in how the object model operates. Especially atomic sessions have a clear point to commit any changes so that many actions can be deferred until all the information is available. In TR-069 there is no real session concept although one RPC must be executed atomically even if it changes multiple parameters. As there are different RPCs to create objects and set their parameters it is impossible to create and parameterize an object in a single session. This creates problems with general DMT models.

It is recommended to operate all RPCs in an atomic session to allow these DMT models to leverage the session commit phase. However, a TR-069 Connector must also accept a read only or exclusive session. The session can then of course cause exceptions to be thrown at certain operations.

The connector must *lazily* create instances. An `addObject(String)` method must not actually create the object, it only has to create an instance id and ensure the uniqueness of this id over time. The id must follow the rules from TR-069, it must not clash with an existing id even after such an id has been used in the past.

This id is then returned to the ACS who will then use it in subsequent RPCs. When one of the subsequent RPCs tries to access this not-yet existent node, for example a get or set, then the TR069 Connector must create it before it sets or gets the value of this node. This lazy strategy allows the node creation and the parameterization of that node to happen in a single session/RPC.

For example, in session 100 the `addObject(String)` creates a new node. This node is not really created but the unique instance id 4311 is assigned to it. After this RPC, the session is closed. The ACS receives this instance and then prepares a `GetParameterValues` RPC to get the `../4311/Foo` parameter. The management agent receives the RPC and opens a new session 200, it then calls `getParameterValue(String)`. The TR069 Connector will not find the appropriate entry 4311 in the table. Instead of raising an error it creates this node and then gets the value for the `../4311/Foo` parameter.

A Data Plugin implementing a native TR-069 object model can override the lazy behavior by adding a `application/x-tr-069-eager` MIME type to the list of MIME types in the Meta Node. If this MIME type is present then the node must be eagerly created during the `addObject(String)` method.

The TR069 Connector must assign the unique id according to the TR-069 rules for instance ids.

131.3.9 Data Types

This specifications assume the [2] *TR-106 Amendment 3* defined data types. TR-106 defines a number of data types, derived from XML Schema and creates a number of sub-types to discriminate between different use cases. A Protocol Adapter must be able to understand the types defined in Table 131.5 to be able to faithfully define a data model based on [2] *TR-106 Amendment 3*. Discriminating between some of the sub-types requires inspection of the data. Each sub-type requires mapping rules that are defined later. Each mapping is assigned a unique MIME sub-type in the application media type. That is, the TR-069 int type has a MIME type of `application/x-tr-069-int`.

Table 131.5 TR-069 Types, MIME types

TR-069 Type	MIME Type	Notes
base64	x-tr-069-base64	Base 64 encoded
hexBinary	x-tr-069-hexBinary	Hex encoded
boolean	x-tr-069-boolean	
string	x-tr-069-string	General string type.
string (list)	x-tr-069-list	A comma separated string that acts as a list.
int	x-tr-069-int	Signed integer

TR-069 Type	MIME Type	Notes
unsignedInt	x-tr-069-unsignedInt	Unsigned integer
long	x-tr-069-long	Signed long
unsignedLong	x-tr-069-unsignedLong	Unsigned long
dateTime	x-tr-069-dateTime	Absolute UTC time, relative boot time, or unknown time
	x-tr-069-eager	Eager creation (not a data type, see <i>Lazy and Sessions</i> on page 19).

It is the responsibility of the Protocol Adapter to properly clean up the parameter values, that is, remove any unnecessary white space, etc. The TR069 Connector must accept any lexically correct form of the value of a parameter. However, the connector must always return the value according to the format of the data types specified by TR-069.

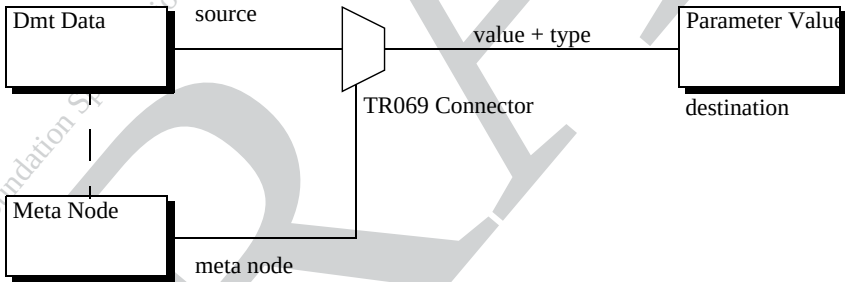
131.3.10

DMT to TR-069 Conversion

This section describes the conversion from a DMT node (a Dmt Data) to a TR-069 Parameter value. The *source* is the DMT node retrieved from the DMT. The *destination* is the value and its type that must be encoded in the TR-069 response. The *meta node* is the Meta Node associated with the source. This model is depicted in Figure 131.7 .

Figure 131.7

DMT to TR-069



The different conversions possible for the Dmt Data to the TR-069 Parameter value are shown in Table 131.6 . This table shows vertically the Dmt Admin formats and horizontally the TR-106 types defined in Table 131.5 . Each row has a default conversion type, indicated with a bold entry. For example, the default conversion of a `???` to the boolean type is the default conversion.

This default conversion can be overridden by the Data Plugin by specifying an alternative MIME type in the list of allowed MIME types in the Meta Node `???`. If this list contains a MIME type that has the prefix `application/x-tr-069-` then the first entry in this list must be chosen as the destination type instead of the default type. This way, a TR-069 Data Plugin can indicate the exact type to a TR-069 Protocol Adapter.

For example, a Dmt Data has the format `???`. However, the Data Plugin for this node has a Meta Node that contains

```
String[] { "application/x-tr-069-hexBinary" }
```

The resulting type must therefore be `hexBinary` in this example.

The Dmt Data nodes are leaf nodes, however, there is a special case for interior LIST nodes marked with a `application/x-tr-069-list` type in the Meta Node. These nodes must be converted to a comma separated string as described in *List* on page 22 .

Cells that are empty in the table indicate an impossible conversion that must be reported. Cells with a name refer to one of the subsequent sections.

Table 131.6 Dmt Data Format to TR-069 Data

	base64	boolean	dateTime	hexBinary	int	long	string	unsignedInt	unsignedLong
???	binary			binary					
???	binary			binary					
???		=					true false		
???			date				=		
???			date				date		
???					number	number	number	number	number
???					number	number	number	number	number
???					number	number	number	number	number
LIST							list		
???		false	date		o	o	"null"	o	o
???	binary			binary					
???							=		
???							=		
???			date				=		
???							=		

131.3.10.1**Date**

If the destination type is string then a date must be formatted according to the TR-069 dateTime format. [???](#) and [???](#) must be set to a [TR069_DATETIME](#) typed destination with just the day or just the time respectively. That is, the [???](#) must be treated as a relative time for TR-069.

The Date object of the Dmt Data object represents the three different [TR069_DATETIME](#) types with the getTime() method. The value of getTime() indicates what type of date time it is:

- *Unknown* - The getTime() method must be 0
- *Relative* - The getTime() method must return a negative number
- *Absolute* - The getTime() method must return a positive number

If a [???](#), [???](#), or [???](#) is converted to a string the string representation of [TR069_DATETIME](#) must be used, including the form of unknown, relative, or absolute. A [???](#) stands for an unknown time.

131.3.10.2**Binary**

The Dmt Admin service has several binary formats ([???](#), [???](#), and [???](#)) that can be converted to [TR069_HEXBINARY](#) and [TR069_BASE64](#). All binary formats maintain their data as a byte[]. Conversion is therefore straightforward encoding of the byte[] into the proper encoding: hex or base 64.

131.3.10.3**Number**

The TR-069 Connector must convert numeric values ([???](#), [???](#), and [???](#)) to [TR069_INT](#), [TR069_LONG](#), [TR069_UNSIGNED_INT](#), and [TR069_UNSIGNED_LONG](#) values. Float values must be rounded according to the standard Java rounding rules when converted to an integer or long.

A conversion must not exceed the range of the destination type. That is, if an integer is converted to an unsigned int then negative values must be treated as an error. If the destination type is string then the numeric value must be calculated with the Dmt Data toString method.

131.3.10.4

List

LIST nodes with primitive children must be converted to a comma separated list. If the children nodes are interior nodes then an error must be raised. The values of the comma separated list must come from the children of the value node. Each of these children must be converted to a string type according to Table 131.6 . These children must then be escaped and concatenated with a comma as separator according to the rules of TR-106 comma separated lists. Nested lists are not allowed.

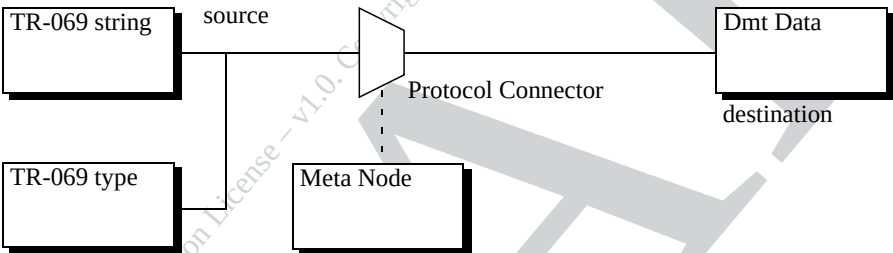
131.3.11

TR-069 to Dmt Data Conversion

A TR-069 Parameter value consists of a string and a type identifier from the set of TR-069 types, see *Data Types* on page 19 . The conversion is depicted in Figure 131.7 .

Figure 131.8

TR-069 to DMT



The destination type is obtained from the corresponding Meta Node. If multiple formats are specified in the result of the ??? method then the most applicable type must be used. The following table lists the applicability for each TR-106 data type.

base64	???, ???, ???
boolean	???, ???
dateTime	???, ???, ???
hexBinary	???, ???, ???
int	???, ???, ???, ???
long	???, ???, ???, ???
string	???, ???, ???, ???,
	???, ???, ???
unsignedInt	???, ???, ???, ???
unsignedLong	???, ???, ???, ???

If the conversion fails and there are untried formats left then the other formats must be used.

There is a special case when the destination node is a LIST node with primitive children and the source is a string type. In that case the string must be parsed according to TR-106 comma separated lists and each element must be stored as a child node.

The conversion matrix is in the following table. The equal sign indicates identity taking into account any encoding. It is not necessary that the source type corresponds to a MIME type in the meta node.

Table 131.7 TR-069 Value to Dmt Data

	???	???	???	???	???	???	???	???	???	???	???	???	LIST
base64	binary	binary							binary				
boolean			bool							true false			
dateTime				date	date					=	date		
hexBinary	binary	binary							binary				
int						num	num	num		=			
long						num	num	num		=			
string			bool			num	num	num	=	=	=	list	
unsignedInt						num	num	num		=			
unsignedLong						num	num	num		=			

- 131.3.11.1 Date**

A [TR069_DATETIME](#) can be converted to a [???](#), [???](#), and [???](#). A [???](#) must take the day part and a [???](#) must take the time part.
- 131.3.11.2 Num**

Source numbers must be converted to their destination counterpart. The conversion result must fail if the result falls outside the range of the destination.
- 131.3.11.3 Bool**

If the source is a string or boolean type and the destination `FORMAT_BOOLEAN` then the conversion must parse the string ignoring the case. The strings `true` and `false` map to their corresponding value. The strings `0` must map to `false` and `1` to `true`.
- 131.3.11.4 Binary**

The source must be decoded according to its TR-069 type ([TR069_BASE64](#) or [TR069_HEXBINARY](#)). The resulting byte array can then be set with the [???](#) with the destination format: [???](#) or [???](#).
- 131.3.11.5 List**

The source is a comma separated list and must be stored as children of the destination node.

131.4 RPCs

The following sections explain in more detail how the different RPCs are supported by the TR069 Connector operate.

131.4.1 Get Parameter Values

The `GetParameterValues` RPC retrieves the value from one or more parameters. Each request in the RPC can request one parameter value or provides an object or table path, requesting multiple values with one path.

The `getParameterValue(String)` method retrieves the value of one parameter in the DMT. The `getParameterNames(String,boolean)` method can be used to retrieve the values of a table or object.

For the `getParameterValue(String)` method the TR069 Connector must first check for synthesized parameters, see *Synthetic Nodes* on page 18 (Alias and NumberOfEntries). Otherwise, the parameter name must be converted to a URI, this must be done according to the `toURI(String,boolean)` method with the boolean set to true, creating any missing nodes if possible. The Dmt Data for this node must be converted according to *DMT to TR-069 Conversion* on page 20 . The returned ParameterValue contains the type and value of the parameter.

For example:

```
ParameterValue v = connector.getParameterValue(
    "Device.DeviceInfo.Manufacturer");
String value = v.getValue();
int type = v.getType();
```

131.4.2

Set Parameter Values

The SetParameterValues RPC sets a number of values in one RPC. The `setParameterValue(String,String,int)` method corresponds to setting a single parameter in the DMT. It takes a parameter path, a value, and the type of this parameter.

The TR069 Connector must first check if the requested destination is the Alias node of a MAP child. If the Alias node is set, the name of the parent node must be renamed to the given value. The value of the Alias node must be a TR-069 string type, the Connector must ensure the value is escaped when necessary. See *Synthetic Nodes* on page 18 for further information about aliases.

Otherwise, the parameter name must be converted to a URI, this must be done according to the `toURI(String,boolean)` method with the boolean set to true.

The given value must be converted to a Dmt Data according to the *TR-069 to Dmt Data Conversion* on page 22 . For example:

```
connector.setParameterValue("Starwars.R2D.2.Start",
    "20110805T10:15:20Z", TRo69_DATETIME );
```

131.4.3

Get Parameter Names

The GetParameterNames RPC allows an ACS to discover the parameters accessible on a particular CPE as well as verifying the existence of a parameter. There are modes for this RPC depending on the path and next level arguments. See the following table.

Table 131.8 Modes based on type of path and NextLevel arguments

NextLevel	Parameter Path	Table or Object Path
true	Invalid Argument Fault code 9003 since this field must always be false for a parameter path.	Include only the children of the object or table.
false	A single ParameterInfo object is returned that provides information about the given parameter.	The whole sub-tree rooted at the given object or table path, this includes the object at the path itself. All objects must be included even if they are empty.

The result must include only parameters, objects, and tables that are actually implemented by the CPE. If a parameter is listed then a `getParameterValue(String)` method called with this parameter's path should succeed. As a convenience, the `ParameterInfo` class provides a `getParameterValue()` method as a short cut to the value.

For example, assume the following instances:

```
IGD.LAN.1.Hosts.
IGD.LAN.1.Hosts.HostNumberOfEntries
IGD.LAN.1.Hosts.Host.
IGD.LAN.1.Hosts.Host.1.
IGD.LAN.1.Hosts.Host.1.Active
IGD.LAN.1.Hosts.Host.2.
IGD.LAN.1.Hosts.Host.2.Active
IGD.LAN.2.Hosts.
IGD.LAN.2.Hosts.HostNumberOfEntries
```

The following table demonstrates some of the different results based on these example instances.

Table 131.9

Example Get Parameter Names

Parameter Name	Next level	Results	Comments
IGD.LAN.1.	false	IGD.LAN.1. IGD.LAN.1.Hosts. IGD.LAN.1.Hosts.HostNumberOfEntries IGD.LAN.1.Hosts.Host. IGD.LAN.1.Hosts.Host.1. IGD.LAN.1.Hosts.Host.1.Active IGD.LAN.1.Hosts.Host.2. IGD.LAN.1.Hosts.Host.2.Active IGD.LAN.1.Hosts.	The path specifies an instance in a table, but since the Next Level is false the whole sub-tree must be returned, including the root of the sub-tree.
	true	IGD.LAN.1.Hosts.	
IGD.LAN.1.Hosts.« 1.Active	false	IGD.LAN.1.Hosts.Host. 1.Active	The path is the same, an instance in a table, but now only the children must be returned for the source. There is only one child, Hosts. This must be returned as an object path.
	true	Fault 9003 Invalid Arguments, next level must be false for a parameter path.	
IGD.LAN.1	false or true	Fault 9003 Invalid Arguments, it is not a parameter path but an instance id	It is not allowed to specify a parameter path that is actual pointing to an instance.

For example:

```
Collection<ParameterInfo> pinfos = connector.getParameterNames("Device.");
for ( ParameterInfo info : pinfos ) {
    if ( info.isParameter() ) {
        System.out.println(
            connector.getParameterValue(info.getName()).getValue());
    }
}
```

131.4.4 Add Object

The AddObject RPC creates a new instance in a table. There basic form for this RPC is to create an object and return the name of this object. It is also possible to specify an alias (a name specified in

square brackets) after the table path. In that case, the alias is used as the node name. In either case, the path must be a valid table path pointing to an existing MAP or LIST node.

When an object is added without an alias then the TR069 Connector must assign a unique id. TR-069 mandates that this id is unique for the table. The TR069 Connector must be able to create and maintain such a persistent id range. The Connector must ensure that any id chosen is not actually already in use or has been handed out recently. How such an id is calculated and maintained is implementation dependent.

If alias based addressing is used, a name between square brackets, then the alias is retrieved from the square brackets. The DMT must then be verified that no node exists in the corresponding table. If it does already exist, an `INVALID_PARAMETER_NAME` exception is thrown. Otherwise the alias is returned as the selected name.

If the corresponding MAP or LIST node has a Meta Node with a MIME type of application/x-tr-69-eager then the alias or instance id must be used to create the node. Otherwise the alias or instance id must be returned without creating the node. The purpose of this lazy creation is to allow a single Set Parameter Values RPC to atomically create a number of nodes and set their values.

For example:

```
String id = connector.addObject( "Starwars.CP.3.Obiwan.");
connector.setParameterValue( "Starwars.CP.3.Obiwan." + id+ ".Name",
                             "cp30", TR069_STRING );
```

The previous code gets an assigned id with the `addObject(String)` method. The `setParameterValue(String,String,int)` then assigns the string cp30 to the Name node. This will first create the actual node since it was not created in the `addObject(String)` method and then sets the value of the DMT Starwars/CP/3/Obiwan/<id>/Name node.

The `addObject(String)` method requires an atomic session. If a non-atomic session is used then the `addObject(String)` method must not attempt to create any objects and an exception must be thrown.

131.4.5 Delete Object

The DeleteObject RPC deletes an object from the tree, it takes the instance path as argument. This behavior is implemented in the `deleteObject(String)` method. The corresponding node must be deleted if it exists. No error must be raised if the node does not exist in the DMT.

For example, deleting the object created in *Add Object* on page 25 :

```
connector.deleteObject("Starwars.CP.3.Obiwan.cp30.");
```

131.5 Error and Fault Codes

The TR069 Connector must translate any Dmt Admin codes into a TR-069 fault code. Since the methods in the `TR069Connector` only relate to a single value it is possible to provide a mapping from Dmt Exception codes to TR-069 fault codes. It is the responsibility of the Protocol Adapter to aggregate these errors in the response to a SetParameterValues RPCs.

A TR069 Connector must prevent exceptions from happening and ensure that the different applicable error cases defined in the TR-069 RPCs are properly reported as a TR069 Exception with the intended fault code. However, this section defines a list of default translations between Dmt Exceptions and TR-069 fault codes.

The following table contains the exceptions and the resulting fault codes. Any obligations that are mandated by the TR-069 protocol are the responsibility of the TR-069 Protocol Adapter. The Dmt Exception is available from the TR-069 Exception for further inspection.

Table 131.10 Exceptions to TR-069 Fault code.

Exception	Fault code	Comments
???	INTERNAL_ERROR	
???	INTERNAL_ERROR	
???	REQUEST_DENIED	
???	INTERNAL_ERROR	
???	INTERNAL_ERROR	
???	REQUEST_DENIED	
???	INVALID_PARAMETER_NAME	
???	RESOURCES_EXCEEDED	
???	INVALID_PARAMETER_TYPE	
???	INTERNAL_ERROR	
???	INVALID_PARAMETER_NAME	
???	NON_WRITABLE_PARAMETER	
???	INTERNAL_ERROR	
???	INTERNAL_ERROR	
???	REQUEST_DENIED	
???	REQUEST_DENIED	
???	REQUEST_DENIED	
???	INVALID_PARAMETER_NAME	
Dmt Illegal State Exception	INTERNAL_ERROR	
Security Exception	REQUEST_DENIED	
Other Exceptions	REQUEST_DENIED	

131.6 Managing the RMT

The RMT is not a native TR-069 model as it is not defined by BBF and it takes advantage of the Dmt Admin features. This section therefore shows a number of examples how the RMT can be managed from an ACS.

For example, on a specific CPE the following bundles are installed, the given name is the location

```
System Bundle
org-apache-felix-webconsole
org-apache-felix-configadmin
org-eclipse-equinox-scr
jp-co-ntt-admin
de-telekom-shell
```

The intention is to:

- Uninstall org-apache-felix-configadmin,
- Install and start org-eclipse-equinox-cm,
- Update jp-co-ntt-admin.

After the successful reconfiguration, the framework must restart. As framework changes must happen in a atomic session, the following parameters must be set in a single RPC:

```
SetParameterValues {
    Framework.Bundle.org-apache-felix-configadmin.RequestedState = UNINSTALLED
    Framework.Bundle.jp-co-ntt-admin.URL = http://....
```

```
Framework.Bundle.org-eclipse-equinox-cm.URL           = http://...
Framework.Bundle.org-eclipse-equinox-cm.RequestedState = ACTIVE
Framework.Bundle.org-eclipse-equinox-cm.AutoStart     = true
Framework.Bundle.System0020Bundle.URL                 = ""
}
```

The Protocol Adapter must open an atomic session on the \$ node as defined in the RMT. It will then set all the parameters in the previous list. As the Framework/Bundle/org-eclipse-equinox-cm node does not exist, the TR069 Connector will create it because it is below a writable MAP node. The System Bundle is updated with an empty string, signalling an update. A System Bundle update is a framework restart.

Once the session is committed after all the SetPropertyValues elements are executed the Data Plugin will perform the actions and report success or failure. The handler must then restart the framework after the commit has returned.

131.7

Native TR-069 Object Models

This section provides an example of a Data Plugin that provides a native TR-069 Object Model. As example is chosen a naive implementation of the Configuration Admin service. The object model implemented has the following definition:

Path	Type	Write	Read	Description
CM.{i}.	Object			
CM.{i}.Pid	string	x	x	The PID
CM.{i}.Properties.{i}.	Object			Property nodes
CM.{i}.Properties.{i}.Key	string	x	x	The key
CM.{i}.Properties.{i}.Value	string	x	x	Comma separated values

The corresponding DMT sub-tree is defined like:

Name	Act	Type	Card.	S	Description
CM	Get	MAP	1	P	Base node for the CM model
[string]	Get Set Add Del	Configuration	o..n	D	A MAP of the PID
InstanceId	Get	int	1	P	The persistent instance Id
Pid	Get	string	1	P	The PID of the configuration
Properties	Get	MAP	1	P	The properties
[string]	Get Set Add Del	LIST	o..n	D	A property definitions; a property consists of a list of strings. Single values are just a list with one element.
[index]	Get Set Add Del	string	o..n	D	An element in the list

The Protocol Adapter allows an ACS to access the data model implemented in the Dmt Plugin. It also allows the creation of new configuration objects.

131.8

org.osgi.service.tr069todmt

Bundles wishing to use this package must list the package in the Import-Package header of the bundle's manifest. This package has two types of users: the consumers that use the API in this package and the providers that implement the API in this package.

Example import for consumers using the API in this package:

```
{@code Import-Package: org.osgi.service.tr069todmt; version="[1.0,2.0)"}
```

Example import for providers of the API in this package:

```
{@code Import-Package: org.osgi.service.tr069todmt; version="[1.0,1.1)"}
```

131.8.1 Summary

- `ParameterInfo` - Maps to the TR-069 {@code ParameterInfoStruct} that is returned from the {@code TR069Connector#getParameterNames(String, boolean)} method.
- `ParameterValue` - Maps to the TR-069 {@code ParameterValueStruct}
- `TR069Connector` - A TR-069 Connector is an assistant to a TR-069 Protocol Adapter developer.
- `TR069ConnectorFactory` - A service that can create TR069 Connector
- `TR069Exception` - This exception is defined in terms of applicable TR-069 fault codes.

131.8.2 public abstract ParameterInfo

Maps to the TR-069 {@code ParameterInfoStruct} that is returned from the {@code TR069Connector#getParameterNames(String, boolean)} method.

131.8.2.1 public abstract ParameterValue getParameterValue() throws TR069Exception

- Provide the value of the node. This method throws an exception if it is called for anything but a parameter

Returns The Parameter Value of the corresponding object

Throws `TR069Exception` – If there is a problem

131.8.2.2 public abstract String getPath()

- The path of the parameter, either a parameter path, an instance path, a table path, or an object path.

Returns The name of the parameter

131.8.2.3 public abstract boolean isParameter()

- Returns {@code true} if this is a parameter, if it returns {@code false} it is an object or table.

Returns {@code true} for a parameter, {@code false} otherwise

131.8.2.4 public abstract boolean isWritable()

- Return {@code true} if this parameter is writable, otherwise {@code false}. A parameter is writable if the `SetParameter` with the given name would be successful if an appropriate value was given. If this is a table path, the method specifies whether or not `AddObject` would be successful. If the parameter path points to a table instance, the method specifies whether or not `DeleteObject` would be successful.

Returns If this parameter is writable

131.8.3 public abstract ParameterValue

Maps to the TR-069 {@code ParameterValueStruct}

131.8.3.1 public abstract String getPath()

- This is the path of a Parameter. In TR-069 this is called the Parameter Name.

Returns The path of the parameter

131.8.3.2 **public abstract int getType()**

- The type of the parameter. One of {@link TR069Connector#TR069_INT}, {@link TR069Connector#TR069_UNSIGNED_INT}, {@link TR069Connector#TR069_LONG}, {@link TR069Connector#TR069_UNSIGNED_LONG}, {@link TR069Connector#TR069_STRING}, {@link TR069Connector#TR069_DATETIME}, {@link TR069Connector#TR069_BASE64}, {@link TR069Connector#TR069_HEXBINARY}. This method is not part of the {@code ParameterValueStruct} but is necessary to encode the type in the XML.

Returns The parameter type

131.8.3.3 **public abstract String getValue()**

- This is the value of the parameter. The returned value must be in a representation defined by the TR-069 protocol.

Returns The value of the parameter

131.8.4 **public abstract TR069Connector**

A TR-069 Connector is an assistant to a TR-069 Protocol Adapter developer. The connector manages the low level details of converting the different TR-069 RPCs to a Device Management Tree managed by Dmt Admin. The connector manages the conversions from the TR-069 Object Names to a node in the DMT and vice versa.

The connector uses a Dmt Session from the caller, which is given when the connector is created. The connector does not implement the exact RPCs but only provides the basic functions to set and get the parameters of an object as well as adding and deleting an object in a table. A TR-069 developer must still parse the XML, handle the relative and absolute path issues, open a Dmt Session etc.

The connector assumes that each parameter or object path is relative to the root of the Dmt Session.

This connector must convert the TR-069 paths to Dmt Admin URIs. This conversion must take into account the {@code LIST} and {@code MAP} concepts defined in the specifications as well as the synthetic parameters {@code NumberOfEntries} and {@code Alias}. These concepts define the use of an {@code InstanceId} node that must be used by the connector to provide a TR-069 table view on the {@code LIST} and {@code MAP} nodes.

131.8.4.1 **public static final String PREFIX = "application/x-tr-069-"**

The MIME type prefix.

131.8.4.2 **public static final int TR069_BASE64 = 64**

Constant representing the TR-069 base64 type.

131.8.4.3 **public static final int TR069_BOOLEAN = 32**

Constant representing the TR-069 boolean type.

131.8.4.4 **public static final int TR069_DATETIME = 256**

Constant representing the TR-069 date time type.

131.8.4.5 **public static final int TR069_DEFAULT = 0**

Constant representing the default or unknown type. If this type is used a default conversion will take place

131.8.4.6 **public static final int TR069_HEXBINARY = 128**

Constant representing the TR-069 hex binary type.

- 131.8.4.7** **public static final int TR069_INT = 1**
Constant representing the TR-069 integer type.
- 131.8.4.8** **public static final int TR069_LONG = 4**
Constant representing the TR-069 long type.
- 131.8.4.9** **public static final String TR069_MIME_BASE64 = "application/x-tr-069-base64"**
Constant representing the TR-069 base64 type.
- 131.8.4.10** **public static final String TR069_MIME_BOOLEAN = "application/x-tr-069-boolean"**
Constant representing the TR-069 boolean type.
- 131.8.4.11** **public static final String TR069_MIME_DATETIME = "application/x-tr-069-dateTime"**
Constant representing the TR-069 date time type.
- 131.8.4.12** **public static final String TR069_MIME_DEFAULT = "application/x-tr-069-default"**
Constant representing the default or unknown type. If this type is used a default conversion will take place
- 131.8.4.13** **public static final String TR069_MIME_EAGER = "application/x-tr-069-eager"**
Constant representing the TR-069 eager type.
- 131.8.4.14** **public static final String TR069_MIME_HEXBINARY = "application/x-tr-069-hexBinary"**
Constant representing the TR-069 hex binary type.
- 131.8.4.15** **public static final String TR069_MIME_INT = "application/x-tr-069-int"**
Constant representing the TR-069 integer type.
- 131.8.4.16** **public static final String TR069_MIME_LONG = "application/x-tr-069-long"**
Constant representing the TR-069 long type.
- 131.8.4.17** **public static final String TR069_MIME_STRING = "application/x-tr-069-string"**
Constant representing the TR-069 string type.
- 131.8.4.18** **public static final String TR069_MIME_STRING_LIST = "application/x-tr-069-string-list"**
Constant representing the TR-069 string list type.
- 131.8.4.19** **public static final String TR069_MIME_UNSIGNED_INT = "application/x-tr-069-unsignedInt"**
Constant representing the TR-069 unsigned integer type.
- 131.8.4.20** **public static final String TR069_MIME_UNSIGNED_LONG = "application/x-tr-069-unsignedLong"**
Constant representing the TR-069 unsigned long type.
- 131.8.4.21** **public static final int TR069_STRING = 16**
Constant representing the TR-069 string type.
- 131.8.4.22** **public static final int TR069_UNSIGNED_INT = 2**
Constant representing the TR-069 unsigned integer type.
- 131.8.4.23** **public static final int TR069_UNSIGNED_LONG = 8**
Constant representing the TR-069 unsigned long type.

131.8.4.24 public abstract String addObject(String path) throws TR069Exception

path A table path with an optional alias at the end

- Add a new node to the Dmt Admin as defined by the AddObject RPC. The path must map to either a {@code LIST} or {@code MAP} node as no other nodes can accept new children.

If the path ends in an alias ({@code [ALIAS]}) then the node name must be the alias, however, no new node must be created. Otherwise, the Connector must calculate a unique instance id for the new node name that follows the TR-069 rules for instance ids. That is, this id must not be reused and must not be in use. That is, the id must be reserved persistently.

If the {@code LIST} or {@code MAP} node has a Meta Node with a MIME type application/x-tr-069-eager then the node must be immediately created. Otherwise no new node must be created, this node must be created when the node is accessed in a subsequent RPC.

The alias name or instance id must be returned as identifier for the ACS.

Returns The name of the new node.

Throws TR069Exception—The following fault codes are defined for this method: 9001, 9002, 9003, 9004, 9005. If an AddObject request would result in exceeding the maximum number of such objects supported by the CPE, the CPE MUST return a fault response with the Resources Exceeded (9004) fault code.

131.8.4.25 public abstract void close()

- Close this connector. This will **not** close the corresponding session.

131.8.4.26 public abstract void deleteObject(String objectPath) throws TR069Exception

objectPath The path to an object in a table to be deleted.

- Delete an object from a table. A missing node must be ignored.

Throws TR069Exception—The following fault codes are defined for this method: 9001, 9002, 9003, 9005. If the fault is caused by an invalid objectPath value, the Invalid Parameter Name fault code (9005) must be used instead of the more general Invalid Arguments fault code (9003). A missing node for {@code objectPath} must be ignored.

131.8.4.27 public abstract Collection<ParameterInfo> getParameterNames(String objectOrTablePath, boolean nextLevel) throws TR069Exception

objectOrTablePath A path to an object or table.

nextLevel If {@code true} consider only the children of the object or table addressed by {@code path}, otherwise include the whole sub-tree, including the addressed object or table.

- Getting the {@link ParameterInfo} objects addressed by path. This method is intended to be used to implement the GetParameterNames RPC.

The connector must attempt to create any missing nodes that are needed for the {@code objectOrTablePath} by using the {@link #toURI(String, boolean)} method with {@code true}.

This method must traverse the sub-tree addressed by the path and return the paths to all the objects, tables, and parameters in that tree. If the nextLevel argument is {@code true} then only the children object, table, and parameter information must be returned.

The returned {@link ParameterInfo} objects must be usable to discover the sub-tree.

If the child nodes have an {@code InstanceId} node then the returned names must include the {@code InstanceId} values instead of the node names.

If the parent node is a {@code MAP}, then the synthetic {@code Alias} parameter must be included.

Any {@code MAP} and {@code LIST} node must include a {@link ParameterInfo} for the corresponding {@code NumberOfEntries} parameter.

- Returns** A collection of {@link ParameterInfo} objects representing the resulting child parameter, objects, and tables as defined by the TR-069 {@code ParameterInfoStruct}.
- Throws** `TR069Exception`– If the fault is caused by an invalid `ParameterPath` value, the Invalid Parameter Name fault code (9005) MUST be used instead of the more general Invalid Arguments fault code (9003). A `ParameterPath` value must be considered invalid if it is not an empty string and does not exactly match a parameter or object name currently present in the data model. If {@code nextLevel} is {@code true} and {@code objectOrTablePath} is a parameter path rather than an object/table path, the method must return a fault response with the Invalid Arguments fault code (9003). If the value cannot be gotten for some reason, this method can generate the following fault codes:

- 9001 {@link TR069Exception#REQUEST_DENIED}
- 9002 {@link TR069Exception#INTERNAL_ERROR}
- 9003 {@link TR069Exception#INVALID_ARGUMENTS}
- 9005 {@link TR069Exception#INVALID_PARAMETER_NAME}

131.8.4.28 **public abstract ParameterValue getParameterValue(String parameterPath) throws TR069Exception**

parameterPath A parameter path (must refer to a valid parameter, not an object or table).

- Getting a parameter value. This method should be used to implement the GetParameterValues RPC. This method does **not** handle retrieving multiple values as the corresponding RPC can request with an object or table path, this method only accepts a parameter path. Retrieving multiple values can be achieved with the {@link #getParameterNames(String, boolean)}.

If the {@code parameterPath} ends in {@code NumberOfEntries} then the method must synthesize the value. The {@code parameterPath} then has a pattern like {@code (object-path)(table-name)NumberOfEntries}. The returned value must be an {@link #TR069_UNSIGNED_INT} that contains the number of child nodes in the table {@code (object-path)(table-name)}. For example, if {@code A.B.CNumberOfEntries} is requested the return value must be the number of child nodes under {@code A/B/C}.

If the value of an {@code Alias} node is requested then the name of the parent node must be returned. For example, if the path is {@code M.X.Alias} then the returned value must be {@code X}.

The connector must attempt to create any missing nodes along the way, creating parent nodes on demand.

Returns The name, value, and type triad of the requested parameter as defined by the TR-069 {@code ParameterValueStruct}.

Throws `TR069Exception`– The following fault codes are defined for this method: 9001, 9002, 9003, 9004, 9005.

- 9001 {@link TR069Exception#REQUEST_DENIED}
- 9002 {@link TR069Exception#INTERNAL_ERROR}
- 9003 {@link TR069Exception#INVALID_ARGUMENTS}
- 9004 {@link TR069Exception#RESOURCES_EXCEEDED}
- 9005 {@link TR069Exception#INVALID_PARAMETER_NAME}

131.8.4.29 **public abstract void setParameterValue(String parameterPath, String value, int type) throws TR069Exception**

parameterPath The parameter path

value A trimmed string value that has the given type. The value can be in either canonical or lexical representation by TR069.

type The type of the parameter ({@link #TR069_INT}, {@link #TR069_UNSIGNED_INT}, {@link #TR069_LONG}, {@link #TR069_UNSIGNED_LONG}, {@link #TR069_STRING}, {@link

#TR069_DATETIME},{@link #TR069_BASE64},{@link #TR069_HEXBINARY},{@link #TR069_BOOLEAN})

- Setting a parameter. This method should be used to provide the SetParameterValues RPC. This method must convert the parameter Name to a URI and replace the DMT node at that place. It must follow the type conversions as described in the specification.

The connector must attempt to create any missing nodes along the way, creating parent nodes on demand.

If the value of an Alias node is set then the parent node must be renamed. For example, if the value of {@code M/X/Alias} is set to {@code Y} then the node will have a URI of {@code M/Y/Alias}. The value must not be escaped as the connector will escape it.

Throws TR069Exception—The following fault codes are defined for this method: 9001, 9002, 9003, 9004, 9005, 9006, 9007, 9008.

- 9001 {@link TR069Exception#REQUEST_DENIED}
- 9002 {@link TR069Exception#INTERNAL_ERROR}
- 9003 {@link TR069Exception#INVALID_ARGUMENTS}
- 9004 {@link TR069Exception#RESOURCES_EXCEEDED}
- 9005 {@link TR069Exception#INVALID_PARAMETER_NAME}
- 9006 {@link TR069Exception#INVALID_PARAMETER_TYPE}
- 9007 {@link TR069Exception#INVALID_PARAMETER_VALUE}
- 9008 {@link TR069Exception#NON_WRITABLE_PARAMETER}

131.8.4.30 **public abstract String toPath(String uri) throws TR069Exception**

uri A Dmt Session relative URI

- Convert a Dmt Session relative Dmt Admin URI to a valid TR-069 path, either a table, object, or parameter path depending on the structure of the DMT. The translation takes into account the special meaning {@code LIST}, {@code MAP}, {@code Alias}, and {@code InstanceId} nodes.

Returns An object, table, or parameter path

Throws TR069Exception—If there is an error

131.8.4.31 **public abstract String toURI(String name, boolean create) throws TR069Exception**

name A TR-069 path

create If {@code true}, create missing nodes when they reside under a MAP or LIST

- Convert a TR-069 path to a Dmt Session relative Dmt Admin URI. The translation takes into account the special meaning {@code LIST}, {@code MAP}, {@code InstanceId} node semantics.

The synthetic {@code Alias} or {@code NumberOfEntries} parameter cannot be mapped and must throw an {@link TR069Exception#INVALID_PARAMETER_NAME}.

The returned path is properly escaped for TR-069.

The mapping from the path to a URI requires support from the meta data in the DMT, it is not possible to use a mapping solely based on string replacements. The translation takes into account the semantics of the MAP and LIST nodes. If at a certain point a node under a {@code MAP} node does not exist then the Connector can create it if the {@code create} flag is set to {@code true}. Otherwise a non-existent node will terminate the mapping.

Returns A relative Dmt Admin URI

Throws TR069Exception—If there is an error

131.8.5 **public abstract TR069ConnectorFactory**

A service that can create TR069 Connector

131.8.5.1 **public abstract TR069Connector create(DmtSession session)**

session The session to use for the adaption. This session must not be closed before the TR069 Connector is closed.

- Create a TR069 connector based on the given session .

The session must be an atomic session when objects are added and/or parameters are going to be set, otherwise it can be a read only or exclusive session. Due to the lazy creation nature of the TR069 Connector it is possible that a node must be created in a read method after a node has been added, it is therefore necessary to always provide an atomic session when an ACS session requires modifying parameters.

Returns A new TR069 Connector bound to the given session

131.8.6 **public class TR069Exception extends RuntimeException**

This exception is defined in terms of applicable TR-069 fault codes. The TR-069 specification defines the fault codes that can occur in different situations.

131.8.6.1 **public static final int INTERNAL_ERROR = 9002**

9002 Internal error

131.8.6.2 **public static final int INVALID_ARGUMENTS = 9003**

9003 Invalid Arguments

131.8.6.3 **public static final int INVALID_PARAMETER_NAME = 9005**

9005 Invalid parameter name (associated with Set/GetParameterValues, GetParameterNames, Set/GetParameterAttributes, AddObject, and DeleteObject)

131.8.6.4 **public static final int INVALID_PARAMETER_TYPE = 9006**

9006 Invalid parameter type (associated with SetParameterValues)

131.8.6.5 **public static final int INVALID_PARAMETER_VALUE = 9007**

9007 Invalid parameter value (associated with SetParameterValues)

131.8.6.6 **public static final int METHOD_NOT_SUPPORTED = 9000**

9000 Method not supported

131.8.6.7 **public static final int NON_WRITABLE_PARAMETER = 9008**

9008 Attempt to set a non-writable parameter (associated with SetParameterValues)

131.8.6.8 **public static final int NOTIFICATION_REJECTED = 9009**

9009 Notification request rejected (associated with SetParameterAttributes method).

131.8.6.9 **public static final int REQUEST_DENIED = 9001**

9001 Request denied (no reason specified)

131.8.6.10 **public static final int RESOURCES_EXCEEDED = 9004**

9004 Resources exceeded (when used in association with SetParameterValues, this MUST NOT be used to indicate parameters in error)

131.8.6.11 public TRo69Exception(String message)*message* The message

- A default constructor when only a message is known. This will generate a {@link #INTERNAL_ERROR} fault.

131.8.6.12 public TRo69Exception(String message, int faultCode, DmtException e)*message* The message*faultCode* The TR-069 defined fault code*e*

- A Constructor with a message and a fault code.

131.8.6.13 public TRo69Exception(String message, int faultCode)*message* The message*faultCode* The TR-069 defined fault code

- A Constructor with a message and a fault code.

131.8.6.14 public TRo69Exception(DmtException e)*e* The Dmt Exception

- Create a TR069Exception from a Dmt Exception.

131.8.6.15 public DmtException getDmtException()*Returns* the corresponding Dmt Exception**131.8.6.16 public int getFaultCode()**

- Answer the associated TR-069 fault code.

Returns Answer the associated TR-069 fault code.

131.9 References

- [1] *TR-069 Amendment 3*
https://www.broadband-forum.org/technical/download/TR-069_Amendment-3.pdf
- [2] *TR-106 Amendment 3*
https://www.broadband-forum.org/technical/download/TR-106_Amendment-3.pdf
- [3] *XML Schema Part 2: Datatypes Second Edition*
<https://www.w3.org/TR/xmlschema-2/>
- [4] *SOAP 1.1*
<https://www.w3.org/TR/soap11/>
- [5] *Extensible Markup Language (XML) 1.0 (Second Edition)*
<https://www.w3.org/TR/xml/#NT-Letter>
- [6] *Broadband Forum*
<https://www.broadband-forum.org/>

End Of Document

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT